

A PROJECT SYNOPSIS ON

RED FORGE

Submitted in partial fulfilment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING



Submitted by

Vighnesh	2303190100033
Rajat Yadav	2303190100019
Ritk Kumar	2303190100022
Vivek Sharma	2303190100037

Under the supervision of

Mrs. Afshan Zameer Khan , Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

RAKSHPAL BAHADUR COLLEGE OF ENGINEERING & TECHNOLOGY

BADAUN ROAD, BAREILLY – 243001, UTTAR PRADESH

Affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow

SESSION 2026 – 27

DECLARATION

We hereby declare that the project synopsis titled “An In-Memory Data Store Implementing a Defined Subset of the Redis Command Set and the RESP2 Protocol” submitted to the Department of Computer Science & Engineering, Rakshpal Bahadur College of Engineering & Technology, Bareilly, is a record of original work carried out by us under the supervision of Mrs. Afshan Zameer Khan. The work presented here has not been submitted elsewhere for the award of any degree or diploma. Every source we have consulted has been acknowledged in the references, and any material reproduced from another work has been cited at the point of use.

We further declare that this synopsis is our own composition. Where we have used an AI assistant, a code generator or any similar tool, its use has been disclosed to our supervisor, and we accept full responsibility for the accuracy of every claim, figure and reference in this document.

Name of Member	Roll No.	Signature
Vighnesh	2303190100033	
Rajat Yadav	2303190100019	
Ritk Kumar	2303190100022	
Vivek Sharma	2303190100037	

Signature of the Guide: _____

Name and Designation: _____

Date: _____

Place: _____

ABSTRACT

Most database systems are studied as black boxes, so the engineering decisions behind them, such as how a server serves thousands of connections on one thread, how expiry survives a restart, or how a crash mid-write is recovered, remain invisible to students. This project opens the box by building an in-memory data store from scratch in the Go programming language that implements a defined subset of the Redis command set over the RESP2 wire protocol, so that the unmodified official Redis client and benchmark tools connect to it directly. The server is a single-threaded event loop over non-blocking TCP sockets, chosen to make every command atomic by construction rather than through locks. It supports strings, lists, hashes and sets, key expiry stored as absolute timestamps with passive and active reclamation, and, as extensions, sorted sets on a skiplist, transactions and publish-subscribe messaging. Durability combines point-in-time binary snapshots with an append-only log of sequence-numbered, checksummed records, recovered by loading the snapshot and replaying only the log entries after its boundary, with defined behaviour for crashes during snapshot writes and for torn records. Asynchronous primary-replica replication provides read scaling and redundancy without claiming automatic failover. The outcome is a working server, a crash-recovery and concurrency test suite, and a controlled five-scenario evaluation against official Redis reporting throughput, tail latency, CPU and memory, with each measured gap attributed in writing to a specific design decision.

Keywords:

In-memory data store; Redis; RESP2 protocol; event-driven server; append-only log; crash recovery; replication

TABLE OF CONTENTS

DECLARATION.....	ii
ABSTRACT.....	iii
LIST OF FIGURES AND TABLES.....	vi
INTRODUCTION.....	1
1.1 Overview of the Domain.....	1
1.2 Motivation.....	1
1.3 Organisation of the Synopsis.....	2
LITERATURE SURVEY.....	3
2.1 Review of Existing Work.....	3
2.2 Comparison of Existing Systems.....	5
2.3 Research Gap.....	5
PROBLEM STATEMENT AND OBJECTIVES.....	7
3.1 Statement of the Problem.....	7
3.2 Objectives.....	7
3.3 Scope and Limitations.....	8
3.4 Success Criteria.....	9
PROPOSED METHODOLOGY.....	11
4.1 Overall Approach.....	11
4.2 Algorithms and Techniques.....	11
4.3 Data Sources.....	13
4.4 Methodology Flow Diagram.....	13
SYSTEM DESIGN.....	15
5.1 System Architecture.....	15
5.2 Data Flow Diagram.....	16
5.3 Database / Data Model.....	17

5.4 Use Case / Sequence Diagram.....	17
HARDWARE AND SOFTWARE REQUIREMENTS.....	19
6.1 Hardware Requirements.....	19
6.2 Software Requirements.....	19
EXPECTED OUTCOME AND EVALUATION.....	20
7.1 Deliverables.....	20
7.2 Evaluation Metrics.....	20
WORK PLAN AND TIMELINE.....	22
8.1 Project Schedule.....	22
8.2 Task Allocation.....	23
CONCLUSION AND FUTURE SCOPE.....	25
9.1 Applications.....	25
9.2 Future Scope.....	25
9.3 Conclusion.....	25
REFERENCES.....	27
TEAM CONTRIBUTION.....	29

LIST OF FIGURES AND TABLES

Figures

Fig. 4.1 Methodology flow	13
Fig. 5.1 System architecture	15
Fig. 5.2 Data flow diagram, level 0	16
Fig. 5.3 Data flow diagram, level 1	16
Fig. 5.4 Data model of the keyspace and persistence files	17
Fig. 5.5 Request lifecycle and recovery sequence	17

Tables

Table 2.1 Comparison of existing systems	5
Table 3.1 Command support matrix	9
Table 6.1 Hardware requirements	19
Table 6.2 Software requirements	19
Table 7.1 Evaluation scenarios	21
Table 8.1 Project schedule	22
Table 8.2 Task allocation	23

CHAPTER 1

INTRODUCTION

1.1 Overview of the Domain

In-memory data stores keep the entire working dataset in main memory and serve reads and writes over a network, trading the capacity of disk for latency measured in microseconds. Redis is the most widely deployed system of this kind and is used as a cache, a session store, a message broker and a primary database for hot data [1]. Its popularity rests on a small number of engineering ideas: a compact text protocol, a single-threaded command loop that makes every operation atomic without locks, a handful of well-chosen data structures, and a persistence design that combines snapshots with an append-only log.

These ideas are not unique to Redis. Memcached introduced the distributed in-memory cache to web applications two decades ago [2], and the RAMCloud project argued that whole datasets should live in DRAM with disks used only for durability [3]. What distinguishes the domain is that correctness and performance depend on low-level systems concerns such as socket handling, syscall overhead, memory allocation and file synchronisation, which most application-level courses never touch.

Studying such a system by reading its source is difficult because production code carries years of optimisation. Rebuilding its core, with a narrow and explicitly declared scope, exposes the same design decisions in a form small enough to reason about, measure and defend.

1.2 Motivation

The team observed that database internals are usually taught as a list of properties rather than as engineering trade-offs that can be measured. We wanted a project in which every design choice has a cost we can put a number on: what a single-threaded loop costs in throughput, what a one-second fsync window costs in durability, and what a blocking keyspace scan costs in stall time.

A second, practical reason is external verifiability. Because the server speaks the real RESP2 protocol [4], the official redis-cli and redis-benchmark tools act as an

independent test harness that the team did not write and cannot bias. A command either works with the real client or it does not.

Finally, the work spans network programming, protocol design, data structure implementation, durability engineering and replication, which together cover the practical core of a systems engineering curriculum in one coherent artefact.

1.3 Organisation of the Synopsis

Chapter 2 surveys Redis, its predecessors and its successors, and the algorithms this project borrows, and identifies the gap the project addresses. Chapter 3 states the problem, the objectives, the scope with its explicit exclusions, and the success criteria. Chapter 4 describes the phased methodology and justifies each algorithmic choice against its alternatives. Chapter 5 presents the system architecture, data flow, data model and request lifecycle. Chapter 6 lists hardware and software requirements. Chapter 7 defines the deliverables and the evaluation methodology. Chapter 8 gives the fourteen-week schedule and the allocation of modules to members. Chapter 9 discusses applications, future scope and conclusions, followed by references and the team contribution page.

CHAPTER 2

LITERATURE SURVEY

2.1 Review of Existing Work

Redis [1], [5], [6]. Redis is an open-source in-memory data structure server. It executes commands on a single thread over an event loop, supports strings, lists, hashes, sets, sorted sets, streams and more, and persists data through RDB snapshots and an append-only file [5]. Replication is asynchronous from a primary to replicas, with Sentinel or Cluster providing failover [6]. Its limitation as a learning reference is its size and maturity: the codebase is large, heavily optimised and mixes many concerns, so the core design is hard to isolate.

Memcached [2]. Fitzpatrick's memcached is a multi-threaded, key-value cache with a simple text protocol and LRU eviction. It scales across cores but offers only opaque string values, no persistence and no replication, so it cannot be used to study durability or recovery.

RAMCloud [3]. Ousterhout et al. proposed keeping all data in DRAM across a cluster with log-structured storage and fast crash recovery by parallel replay. The design shows that memory-resident storage can be durable, but it is a distributed research system that depends on low-latency networking and is far beyond the scale of a student project.

KeyDB [7]. KeyDB is a multi-threaded fork of Redis that runs several event loops in parallel and uses fine-grained locking around the keyspace. It gains throughput on multi-core machines at the cost of reintroducing lock ordering and contention, exactly the complexity the single-threaded design avoids.

Dragonfly [8]. Dragonfly reimplements the Redis and Memcached protocols on a shared-nothing, multi-threaded architecture that partitions the keyspace across cores and uses a novel dashtable. It demonstrates that protocol compatibility can be achieved by an independent implementation, but its complexity is high and its transaction protocol is research-level.

Valkey [9]. Valkey is the Linux Foundation fork of Redis created after the 2024 licence change. It preserves the Redis architecture and protocol while adding I/O threading. It confirms that RESP is a de facto standard worth targeting, but as a production system it shares the same barrier to study as Redis itself.

Skip lists [10]. Pugh's skip list is a probabilistic ordered structure that gives expected logarithmic search, insert and delete with far simpler code than balanced trees. Redis uses it for sorted sets. Its limitation is that its guarantees are expected rather than worst-case, and level assignment must be implemented carefully.

ARIES [11]. Mohan et al. formalised write-ahead logging with log sequence numbers, checkpoints and idempotent redo. This project borrows the sequence-number boundary between snapshot and log directly from this work. ARIES itself targets disk-resident databases with undo logging and is heavier than an in-memory store needs.

The C10K problem [12]. Kegel's survey catalogued the strategies for serving ten thousand concurrent connections, establishing non-blocking sockets with readiness notification as the standard approach. It is a design guide rather than a system and predates modern kernel interfaces, but its core argument motivates the event loop used here.

Redis in Action [13]. Carlson's book documents the practical usage patterns of Redis and the reasons its data types are shaped as they are. It is written for users rather than implementers, so it does not cover the internals this project rebuilds.

2.2 Comparison of Existing Systems

Ref.	Author and Year	Technique Used	Limitation
[1], [5], [6]	Redis Ltd., 2024	Single-threaded event loop, RDB + AOF persistence, async replication	Large, heavily optimised codebase; core design hard to isolate for study
[2]	B. Fitzpatrick, 2004	Multi-threaded cache, text protocol, LRU eviction	No persistence, no replication, opaque values only
[3]	J. Ousterhout et al., 2009	Cluster-wide DRAM storage with log-structured durability	Distributed research system; needs low-latency networking; beyond project scale
[7]	KeyDB, 2023	Multiple event loops with fine-grained locking	Reintroduces lock contention and ordering complexity
[8]	DragonflyDB, 2024	Shared-nothing multi-threaded architecture, dashtable	High complexity; research-level transaction protocol
[9]	Valkey, 2024	Redis architecture with I/O threading	Same barrier to study as Redis
[10]	W. Pugh, 1990	Probabilistic skip list	Expected rather than worst-case bounds
[11]	C. Mohan et al., 1992	Write-ahead logging with LSNs and checkpoints	Designed for disk-resident databases with undo logging
[12]	D. Kegel, 2006	Non-blocking sockets with readiness notification	Design survey, not a system; predates modern kernel interfaces

Table 2.1 Comparison of existing systems

2.3 Research Gap

None of the reviewed work provides a small, self-contained implementation in which the core decisions of an in-memory server, namely the single-threaded event loop,

absolute-timestamp expiry, snapshot-plus-log recovery with a defined torn-record policy, and asynchronous replication, are built with a declared scope, tested for crash-recovery properties, and measured against the production reference so that each performance gap is attributed to a specific design choice. The production systems are too large to study directly, the research systems solve a different problem at a different scale, and the algorithmic works describe components rather than a whole server. This is the gap the present project addresses, and Chapter 3 states it as a problem with checkable objectives.

CHAPTER 3

PROBLEM STATEMENT AND OBJECTIVES

3.1 Statement of the Problem

Widely used in-memory data stores expose a simple command interface, yet the mechanisms that make them fast, atomic and durable are hidden inside large production codebases that students cannot realistically read, modify or measure. As a result, the engineering trade-offs that define such a system, including how a single execution thread serves many concurrent clients without locks, how key expiry remains correct across a restart, how a point-in-time snapshot and an append-only log are reconciled after a crash, how partially written records are handled, and what replication does and does not guarantee, are learned as statements rather than as verified, quantified facts. There is no compact reference implementation with a declared scope whose correctness can be checked by an independent client and whose performance can be compared under controlled conditions against the production system it models. The problem is to build and evaluate such an implementation.

3.2 Objectives

1. Implement a RESP2 protocol codec and a single-threaded, non-blocking TCP event loop such that the unmodified official Redis client connects and issues commands with no custom client code.
2. Implement the string, key expiry, list, hash and set command subset listed in Table 3.1, with expiry stored as absolute timestamps and reclaimed both passively on access and actively by a background sweep.
3. Implement hybrid persistence: an append-only log of sequence-numbered, checksummed records with a configurable fsync policy, and atomic binary snapshots, with startup recovery that replays only log records after the snapshot boundary.
4. Build a correctness test suite covering state invariants, crash recovery under SIGKILL, expiry across restart, and 100 concurrent clients, runnable from a clean checkout with one command.

5. Evaluate the server against official Redis under a controlled methodology across five workload scenarios, reporting throughput, p50/p95/p99/p99.9 latency, CPU and memory, and attribute each gap to a specific architectural decision.
6. Extend the system, in priority order and only after the above are complete, with asynchronous primary-replica replication, skiplist-backed sorted sets, log compaction, transactions and publish-subscribe.

3.3 Scope and Limitations

Scope. The project delivers a single-node server in Go implementing the commands in Table 3.1. Items marked **Must** define project success; items marked **Stretch** are attempted only after every **Must** item is merged and are cut, in the order Pub/Sub, transactions, log compaction, sorted sets, replication, if the schedule slips. The word **compatible** is used in a bounded sense: compatible with RESP2 and with the enumerated command subset, sufficient for unmodified Redis tooling to operate within that subset. It is not a claim of general Redis compatibility.

Limitations. The following are explicitly out of scope, so that their absence is not mistaken for an oversight: cluster mode and consistent hashing, so there is no horizontal write scaling; Sentinel-style automatic failover, so replication provides redundancy but not high availability; the RESP3 protocol; access control lists and TLS, so the server is not deployable on an untrusted network; Lua scripting; streams, HyperLogLog, geospatial and bitmap types; and cursor-based SCAN iteration, so KEYS is the only keyspace enumeration primitive. Throughput is bounded by one core by design, and any $O(N)$ command blocks all other clients for its duration. Expiry is tied to wall-clock time, so a backwards clock adjustment can extend key lifetimes; this is documented rather than mitigated.

Table 3.1 lists every command with its status and its complexity in this implementation.

Category	Commands	Complexity	Status
Connection / Server	PING, ECHO, INFO, COMMAND	O(1)	Must
Server	BGSAVE	O(N) background	Must
Strings	SET, GET, INCR	O(1)	Must
Strings / Keyspace	DEL, EXISTS	O(1) per key	Must
Keyspace	KEYS	O(N) blocking	Must
Expiration	EXPIRE, TTL, PERSIST	O(1)	Must
Lists	LPUSH, RPUSH, LPOP, RPOP, LLEN	O(1)	Must
Lists	LRANGE	O(S+N)	Must
Hashes	HSET, HGET, HDEL, HLEN	O(1) average	Must
Hashes	HGETALL	O(N)	Must
Sets	SADD, SREM, SISMEMBER	O(1) average	Must
Sets	SMEMBERS	O(N)	Must
Sets	SINTER	O(N*M) worst case	Must
Sorted sets	ZADD, ZSCORE, ZRANK	O(log N)	Stretch
Sorted sets	ZRANGE	O(log N + M)	Stretch
Transactions	MULTI, EXEC, DISCARD	O(N) queued	Stretch
Pub/Sub	SUBSCRIBE, UNSUBSCRIBE, PUBLISH	O(1) / O(subscribers)	Stretch
Replication	REPLICAOF	O(N) initial sync	Stretch
Excluded	SCAN, EVAL, XADD and family, HELLO 3	-	Excluded

Table 3.1 Command support matrix

3.4 Success Criteria

1. Objective 1 is met when an unmodified redis-cli performs a full session covering every Must command, and 100 concurrent clients are served without one goroutine per connection.

2. Objective 2 is met when every command in Table 3.1 marked Must passes its invariant tests, and a key with a TTL that is snapshotted, left offline longer than its TTL and restarted is absent, while one restarted sooner reports a TTL reduced by the offline interval within one second.
3. Objective 3 is met when all four crash-recovery tests pass: recovery from the log alone, from snapshot plus log tail, after SIGKILL during BGSAVE with no data loss, and after a hand-truncated log record with replay stopping cleanly at the last valid record.
4. Objective 4 is met when the full suite passes from a clean checkout with a single command, and 100 clients issuing INCR N times each to one key yield exactly 100 times N.
5. Objective 5 is met when all five scenarios of Table 7.1 complete on both servers, three runs each after a warm-up, with the comparison table and a written attribution for every gap; the KEYS stall time is additionally measured at 10 000, 100 000 and 1 000 000 keys.
6. Objective 6 is met, for replication, when a write on the primary is visible on two replicas and a write to a replica is rejected; each other extension is judged by its command tests.

CHAPTER 4

PROPOSED METHODOLOGY

4.1 Overall Approach

The work proceeds in four phases governed by a cut line agreed before any code is written. Phase 0 (weeks 1 to 2) is a shared foundation in which all four members build together the RESP2 codec, the non-blocking listener and event loop, the command dispatcher, PING and ECHO, and the write sequence-number counter; at the end of week 2 the store interface, handler signature and sequence-number contract are frozen so that the tracks can proceed independently. Phase 1 (weeks 3 to 8) runs four parallel tracks: core engine and expiry; lists, hashes and sets followed by the benchmark harness; persistence; and protocol hardening with crash-recovery tests and tooling. The week 8 checkpoint verifies that every Must item is merged. Phase 2 (weeks 9 to 11) is replication, attempted only if the checkpoint passes; otherwise all four members finish outstanding Must items. Phase 3 (weeks 12 to 14) is reserved for evaluation, hardening, documentation and rehearsal of the demonstration and cannot be encroached upon.

Inside the server, the sequence from input to output is: bytes arrive on a non-blocking socket, the event loop hands them to the RESP2 parser, the parser produces an argument vector, the dispatcher invokes the handler, the handler mutates the store and appends a sequence-numbered record to the log, and the reply is serialised back onto the socket.

4.2 Algorithms and Techniques

Single-threaded event loop over non-blocking sockets. We chose one execution thread because it eliminates lock contention on the shared keyspace and makes every command atomic by construction, so there is no lock ordering, no deadlock class and no contention benchmark. We rejected one goroutine per connection because it would require a mutex around the store, and a multi-loop design like KeyDB because the concurrency model itself is the object of study. The cost, throughput bounded by one core, is quantified in the evaluation rather than hidden.

Absolute expiration timestamps. Each key with a TTL stores an absolute expiry time in Unix milliseconds. We rejected storing the remaining duration because a snapshot taken with 100 seconds remaining, followed by 50 seconds offline, would resurrect 50 seconds of lifetime on restart. Absolute timestamps make the log and snapshot time-independent, so EXPIRE is rewritten into the log as a PEXPIREAT-style record. Reclamation is two-tier: passive deletion on lookup, and an active background sweep that samples the expiring-key set, because passive alone leaks untouched keys and active alone leaves a window in which expired keys are visible.

Sequence-numbered append-only log with snapshot boundary. Every write receives a monotonically increasing sequence number and the snapshot records the highest one it contains, following the log-sequence-number idea of ARIES [11]. Replay skips records at or below the boundary, so the two sources cannot disagree and replay is idempotent. We rejected timestamps as the boundary because clocks are not monotonic, and we rejected truncating the log at snapshot time because a crash between the two steps would lose data.

Atomic snapshot by temp file and rename. BGSAVE writes to a temporary file, fsyncs, and renames it into place, so a crash mid-write leaves the previous snapshot intact and an orphan that is deleted at the next startup. We rejected writing in place because a torn snapshot would be unrecoverable.

Length-prefixed, checksummed log records. A record that is truncated or fails its checksum terminates replay and the file is truncated to the last valid boundary, the standard write-ahead-log torn-write policy: the write was never acknowledged as durable, so discarding it is correct. The fsync policy is configurable as always, everysec or no, defaulting to everysec, and the durability cost of always is measured in the evaluation.

Hash map keyspace, doubly linked lists, hash-backed sets, skiplist sorted sets. Go's built-in map gives $O(1)$ average lookup for the keyspace, hashes and sets. Lists use a deque for $O(1)$ pushes and pops at both ends. Sorted sets use Pugh's skip list [10] because it gives $O(\log N)$ rank and range queries with much simpler code than a balanced tree; we rejected a sorted slice because insertion would be $O(N)$.

KEYS retained despite its blocking cost. KEYS is $O(N)$ and blocks every other client under the single-threaded model. We keep it because client tooling and the test suite depend on keyspace enumeration, and because measuring its stall time turns the limitation into a quantified result. SCAN, the cursor-based alternative, is documented as future work.

Go as the implementation language. Go provides a non-blocking network stack in its standard library [14], a garbage collector that removes one class of memory bug, and fast builds, while still exposing syscall-level behaviour. We rejected C because memory safety errors would consume schedule, and rejected Python because its interpreter overhead would dominate every measurement.

4.3 Data Sources

The project does not use a pre-existing dataset. All data is generated during testing and evaluation. The correctness suite generates keys and values programmatically to exercise each invariant, including keyspaces of 10 000 to 1 000 000 keys for the KEYS stall measurement. The evaluation uses the workloads generated by the official redis-benchmark tool [15]: 64-byte and 4 KB values, 50 clients, pipeline depths of 1 and 16, and a client sweep at 1, 10, 50 and 200 connections, as fixed in Table 7.1. The reference system is the official Redis server, whose source is available under the Redis Source Available License and which is used unmodified as a benchmark target only. The team's own code is published under the MIT licence. No personal or licensed data is involved, and no pre-processing is required beyond flushing the keyspace between runs.

4.4 Methodology Flow Diagram

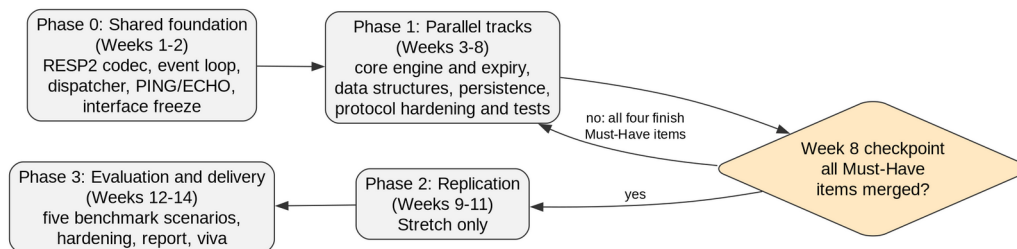


Fig. 4.1 Methodology flow

Fig. 4.1 shows the four phases and the single decision point that governs them. Phase 0 produces the shared foundation and freezes the interfaces on which every later track depends; the exit criterion is that redis-cli PING returns PONG on every member's machine. Phase 1 splits into four parallel tracks that merge to the main branch at least weekly, with a live demonstration against redis-cli at every weekly sync. The week 8 checkpoint is the diamond: if any Must item is not merged, the loop returns to Phase 1 with all four members reassigned to it and replication is cut; only if every Must item is merged does Phase 2 begin. Phase 3 is entered from either path, because evaluation and delivery are where the grade is earned and are never allowed to slip.

CHAPTER 5

SYSTEM DESIGN

5.1 System Architecture



Fig. 5.1 System architecture

Fig. 5.1 shows the modules and the data flowing between them. Clients connect over TCP; the sockets are non-blocking and are multiplexed by a single event loop. Bytes are decoded by the RESP2 parser into an argument vector, which the dispatcher routes to a command handler. All handlers operate on one shared in-memory store holding strings, lists, hashes, sets and, as an extension, sorted sets. Three subsystems hang off the store: the expiry module, which deletes expired keys passively on access and actively by a periodic sweep; the append-only log writer, which records every write with a sequence number; and the snapshot writer, which serialises the whole store on BGSAVE through a temporary file and atomic rename. The dashed replication module, an extension, streams the same sequence-numbered writes to replicas. Because there is one thread, no module needs a lock on the store.

5.2 Data Flow Diagram

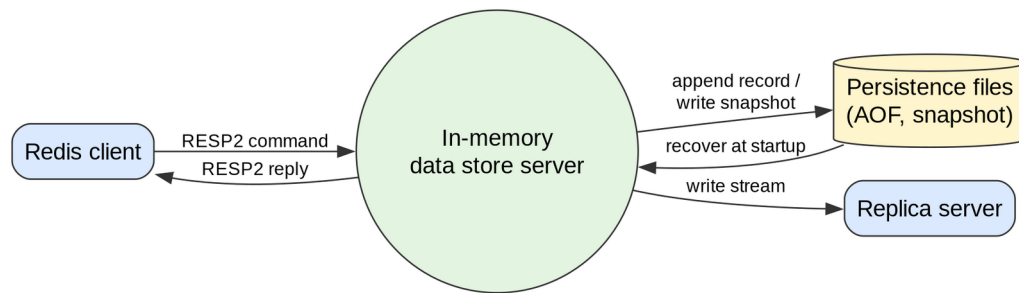


Fig. 5.2 Data flow diagram, level 0

At level 0 the server is one process exchanging RESP2 commands and replies with clients, appending records to and writing snapshots into the persistence files, recovering from those files at startup, and streaming writes to a replica.

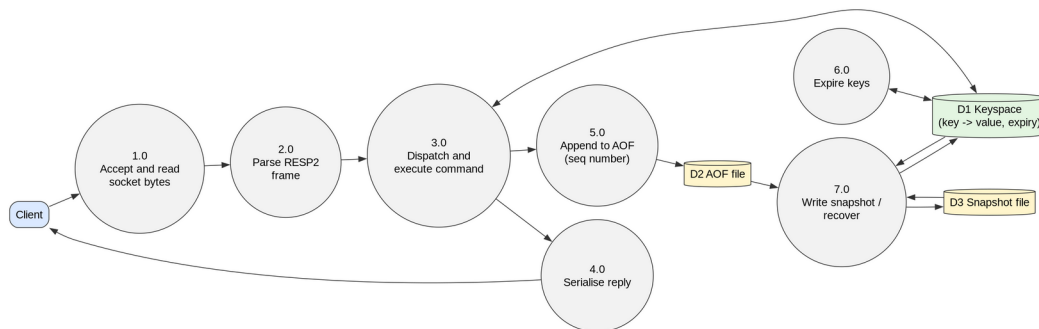


Fig. 5.3 Data flow diagram, level 1

Level 1 decomposes the server into seven processes. Process 1.0 accepts connections and reads socket bytes; 2.0 parses RESP2 frames; 3.0 dispatches and executes the command against the keyspace store D1 and hands each write to 5.0, which appends a sequence-numbered record to the log D2; 4.0 serialises the reply. Process 6.0 is the expiry sweep, which reads and deletes from D1 on a timer. Process 7.0 writes the snapshot D3 on BGSAVE and, at startup, rebuilds D1 from D3 and the tail of D2.

5.3 Database / Data Model

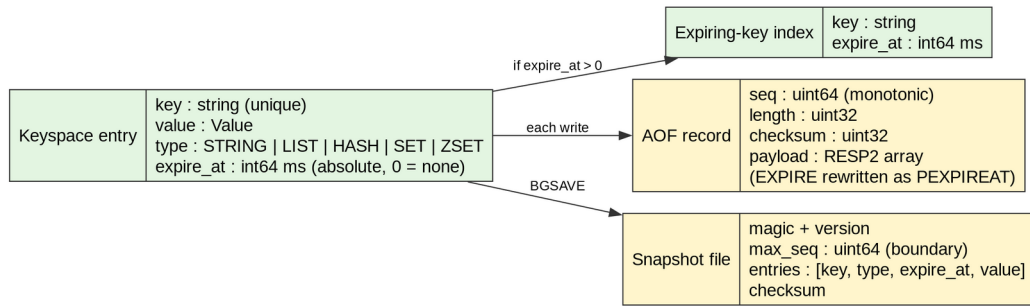


Fig. 5.4 Data model of the keyspace and persistence files

The store is a single hash map from key to entry. Each entry carries a type tag, the typed value, and an absolute expiry time in milliseconds, zero meaning no expiry. Keys with an expiry are also indexed in a separate expiring-key set that the active sweep samples. Each log record holds a monotonic sequence number, a length prefix, a checksum and the command as a RESP2 array, with EXPIRE rewritten in its absolute PEXPIREAT form. The snapshot file holds a magic number and version, the highest sequence number it contains, one entry per key with its type, expiry and value, and a trailing checksum. The sequence number is the join between the two files: recovery loads the snapshot and replays only records whose sequence exceeds the snapshot boundary.

5.4 Use Case / Sequence Diagram

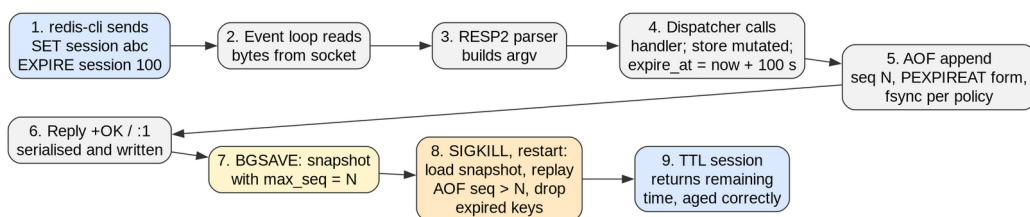


Fig. 5.5 Request lifecycle and recovery sequence

Fig. 5.5 traces the representative workflow used in the final demonstration. A client issues SET followed by EXPIRE. The event loop reads the bytes, the parser builds the argument vector, and the dispatcher invokes the handler, which stores the value and records an absolute expiry time. The write is appended to the log with sequence number N in PEXPIREAT form and fsynced according to policy, and the reply is serialised back to the client. A BGSAVE then produces a snapshot whose boundary is N. The server is killed with SIGKILL and restarted; it loads the snapshot, replays only

records after N, drops keys that expired while offline, and begins serving. A final TTL query returns the remaining lifetime reduced by the offline interval, demonstrating snapshot-plus-log recovery and correctly aged expiry in one sequence.

CHAPTER 6

HARDWARE AND SOFTWARE REQUIREMENTS

6.1 Hardware Requirements

Component	Specification
Processor	x86-64, 4 cores or more (development and benchmarking on the same machine)
RAM	8 GB minimum; 16 GB recommended for the 1 000 000-key stall measurement
Storage	20 GB free on an SSD (fsync latency dominates the write benchmarks)
GPU (if required)	Not required
Deployment target	Single Linux host or Docker Compose topology of one primary and two replicas

Table 6.1 Hardware requirements

6.2 Software Requirements

Category	Tool / Technology and Version
Operating system	Linux (Arch Linux or Ubuntu 22.04 LTS), kernel 6.x
Language	Go 1.22 or later
Framework / libraries	Go standard library only (net, os, encoding/binary, hash/crc32, testing)
Database	None external; the project is the data store. Official Redis 7.x as the evaluation baseline
External APIs	None. RESP2 protocol specification; redis-cli and redis-benchmark 7.x as clients
Development tools	Git and GitHub with pull-request review, GitHub Actions CI, Docker and Docker Compose, Graphviz for diagrams, gofmt and go vet

Table 6.2 Software requirements

CHAPTER 7

EXPECTED OUTCOME AND EVALUATION

7.1 Deliverables

1. A server binary built from a single command that speaks RESP2 and implements every Must command of Table 3.1, with configurable port, persistence directory and fsync policy.
2. The persistence layer: append-only log with checksummed, sequence-numbered records; atomic binary snapshots via BGSAVE; startup recovery from snapshot plus log tail with torn-record truncation and orphaned-temp cleanup, surfaced through INFO.
3. A correctness test suite, runnable from a clean checkout with one command and executed in CI on every pull request, covering state invariants for each data type, four SIGKILL crash-recovery cases, four expiry-across-restart cases, and three concurrency cases with 100 clients.
4. A benchmark harness that scripts the five scenarios, captures the machine specification, runs each scenario three times after a warm-up, and reports median and spread.
5. An evaluation report with comparison tables and charts against official Redis and a written attribution of each gap to a specific design decision.
6. Documentation: README with build and run instructions, the four architecture diagrams (system, persistence flow, request lifecycle and, if built, replication flow), a Docker Compose file for the demonstration topology, and the final report.
7. Extensions, if the schedule allows and in this order: replication, sorted sets, log compaction, transactions and Pub/Sub.

7.2 Evaluation Metrics

Correctness is measured by the test suite: every objective in Section 3.4 has a pass or fail condition, and the suite must be fully green from a clean checkout. Performance is

measured by a controlled comparison against the official Redis server as baseline, both running on the same machine in the same session, bound to loopback to remove network variability, with persistence configuration stated per scenario and matched across the two servers. The machine's CPU model and core count, RAM, OS and kernel version, Go version and Redis version are recorded. Each scenario is run three times after a discarded warm-up run, the keyspace is flushed between runs, and the median is reported with the spread. The metrics captured per scenario are requests per second, average latency, p50, p95, p99 and p99.9 latency, peak CPU utilisation and peak resident memory. Table 7.1 lists the scenarios.

No.	Scenario	Purpose
1	Pure GET, 64-byte values, 50 clients, no pipelining	Read path baseline
2	Pure SET, 64-byte values, 50 clients, no pipelining	Write path baseline including log cost
3	Mixed 80% GET / 20% SET, 64-byte values, 50 clients	Realistic cache workload
4	Pure SET, 4 KB values, 50 clients	Isolates payload size and allocator behaviour
5	Pure SET, 64-byte values, pipeline depth 16, client sweep at 1, 10, 50, 200; repeated at fsync always vs everysec	Isolates syscall overhead; locates event-loop saturation; quantifies durability cost

Table 7.1 Evaluation scenarios

A supplementary measurement records event-loop stall time during KEYS against keyspaces of 10 000, 100 000 and 1 000 000 keys. The report attributes each observed gap to a specific decision rather than to general slowness; candidate attributions include Go's garbage collector versus manual memory management, the absence of a small-object allocator, the absence of compact encodings such as listpack for small collections, and per-command allocation in the RESP codec.

CHAPTER 8

WORK PLAN AND TIMELINE

8.1 Project Schedule

Phase	Activity	Timeline
I	Shared foundation: RESP2 codec, event loop, dispatcher, PING/ECHO, sequence counter; interface freeze; first architecture and request-lifecycle diagrams	Weeks 1 to 2 (Sep 2026)
II	Parallel tracks: core engine and expiry; lists, hashes, sets; log and snapshot persistence; protocol edge cases, CI, crash-recovery tests, Docker; benchmark harness in week 7	Weeks 3 to 8 (Sep to Oct 2026)
III (mid-term review)	Week 8 checkpoint: all Must items except evaluation and documentation merged; benchmark harness runs; live demo against redis-cli	Week 8 (end Oct 2026)
IV	Replication (Stretch): REPLICAOF, handshake, write propagation, replica apply loop, read-only enforcement; or completion of any outstanding Must item	Weeks 9 to 11 (Nov 2026)
V (final review)	Evaluation of five scenarios against official Redis; hardening and packaging; README, diagrams, attribution analysis, future work; two full rehearsals of the demonstration	Weeks 12 to 14 (Nov to Dec 2026)

Table 8.1 Project schedule

8.2 Task Allocation

Member Name	Roll No.	Module / Responsibility	Phase
Vighnesh	2303190100033	Core engine, keypace store, expiry, INFO/COMMAND; transactions (Stretch); replication handshake (Stretch); code review and merges	0, I, II, IV
Rajat Yadav	2303190100019	Persistence: append-only log, snapshots, recovery hardening; log compaction (Stretch); write propagation (Stretch)	0, I, II, IV
Ritk Kumar	2303190100022	Lists, hashes, sets; benchmark harness and evaluation lead; sorted sets (Stretch); replica apply loop (Stretch)	0, I, II, IV, V
Vivek Sharma	2303190100037	RESP2 edge cases, test runner and CI, crash-recovery tests, Docker Compose; pipelining and Pub/Sub (Stretch); replication testing and diagram (Stretch)	0, I, II, IV, V

Table 8.2 Task allocation

Every module has a primary who implements it and a secondary reviewer who approves every pull request and must be able to explain the module without the primary present: core engine (Vighnesh, reviewed by Rajat Yadav), data structures (Ritk Kumar, reviewed by Vighnesh), persistence (Rajat Yadav, reviewed by Ritk Kumar), protocol and Pub/Sub (Vivek Sharma, reviewed by Vighnesh), recovery tests and tooling (Vivek Sharma, reviewed by Rajat Yadav), evaluation (Ritk Kumar, reviewed by Rajat Yadav), and replication (Vighnesh, reviewed by Ritk Kumar). At each weekly sync one module is chosen at random and its secondary presents it, so

viva rehearsal begins in week 3. All four members work together in Phases 0, IV and V.

CHAPTER 9

CONCLUSION AND FUTURE SCOPE

9.1 Applications

The server can realistically serve three settings. First, as a session and cache store for a small web application, where the working set fits in memory, the string and hash commands cover the access pattern, and the append-only log gives restart durability. Second, as a teaching and laboratory artefact in an operating systems or database internals course, where students can read the whole implementation, break the persistence layer deliberately and observe recovery, and reproduce the benchmark methodology. Third, as a lightweight leaderboard or counter service inside a game or classroom application, using INCR and, if built, the sorted-set commands, where a single node with replicas for read scaling is sufficient.

9.2 Future Scope

Cluster mode is the most defensible omission and the first extension the team would build: partitioning the keyspace across nodes with consistent hashing and MOVED and ASK redirects would give horizontal write scaling. A cursor-based SCAN command would remove the blocking cost of KEYS by returning a bounded number of keys per call and letting the event loop serve other clients between calls. Automatic failover, promoting a replica when the primary is lost as Sentinel does, would turn replication into real high availability. Further candidates are RESP3 negotiation and a monotonic-clock scheme for expiry that is immune to wall-clock adjustments.

9.3 Conclusion

The internals of in-memory data stores are widely used but rarely understood in a form that can be inspected and measured. This project rebuilds the core of one such system in Go with a declared command subset over the RESP2 protocol, so that the official Redis client verifies its correctness independently. The single-threaded event loop, absolute-timestamp expiry, sequence-numbered log with atomic snapshots and asynchronous replication are each implemented with defined guarantees and tested under crash and concurrency conditions. The expected contribution is a working,

documented server together with a controlled evaluation that attributes every measured difference from the production system to a specific design decision.

REFERENCES

- [1] Redis Ltd., “Redis documentation.” Accessed: Sep. 3, 2026. [Online]. Available: <https://redis.io/docs/latest/>
- [2] B. Fitzpatrick, “Distributed caching with memcached,” *Linux Journal*, no. 124, Aug. 2004.
- [3] J. Ousterhout et al., “The case for RAMClouds: Scalable high-performance storage entirely in DRAM,” *ACM SIGOPS Oper. Syst. Rev.*, vol. 43, no. 4, pp. 92–105, Dec. 2009.
- [4] Redis Ltd., “Redis serialization protocol specification.” Accessed: Sep. 3, 2026. [Online]. Available: <https://redis.io/docs/latest/develop/reference/protocol-spec/>
- [5] Redis Ltd., “Redis persistence.” Accessed: Sep. 3, 2026. [Online]. Available: https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/
- [6] Redis Ltd., “Redis replication.” Accessed: Sep. 3, 2026. [Online]. Available: https://redis.io/docs/latest/operate/oss_and_stack/management/replication/
- [7] KeyDB, “KeyDB documentation.” Accessed: Sep. 3, 2026. [Online]. Available: <https://docs.keydb.dev/>
- [8] DragonflyDB, “Dragonfly documentation.” Accessed: Sep. 3, 2026. [Online]. Available: <https://www.dragonflydb.io/docs>
- [9] Valkey contributors, “Valkey documentation.” Accessed: Sep. 3, 2026. [Online]. Available: <https://valkey.io/docs/>
- [10] W. Pugh, “Skip lists: A probabilistic alternative to balanced trees,” *Commun. ACM*, vol. 33, no. 6, pp. 668–676, Jun. 1990.
- [11] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, “ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging,” *ACM Trans. Database Syst.*, vol. 17, no. 1, pp. 94–162, Mar. 1992.
- [12] D. Kegel, “The C10K problem.” Accessed: Sep. 3, 2026. [Online]. Available: <http://www.kegel.com/c10k.html>
- [13] J. L. Carlson, *Redis in Action*. Shelter Island, NY, USA: Manning, 2013.

- [14] The Go Authors, “Package net, Go standard library.” Accessed: Sep. 3, 2026. [Online]. Available: <https://pkg.go.dev/net>
- [15] Redis Ltd., “Redis benchmark.” Accessed: Sep. 3, 2026. [Online]. Available: https://redis.io/docs/latest/operate/oss_and_stack/management/optimization/benchmarks/

TEAM CONTRIBUTION

Name the module each member owns. Your guide uses this page to attribute work during the review, so fill it in honestly and make sure every member signs.

S.No	Member Name	Roll No.	Module / Responsibility
1	Vighnesh	2303190100033	Core engine, expiry, transactions, replication handshake; lead reviewer
2	Rajat Yadav	2303190100019	Persistence: append-only log, snapshots, recovery; write propagation
3	Ritk Kumar	2303190100022	Lists, hashes, sets, sorted sets; benchmark harness and evaluation
4	Vivek Sharma	2303190100037	Protocol hardening, test runner and CI, crash-recovery tests, Docker tooling, Pub/Sub